CS361 - HW2

Student: Shrey Poshiya

Github Link:

> **Problem 1**
>
> In the classic problem of the **Towers of Hanoi**, you have 3 towers/rods and n disks of different sizes which can slide onto any tower. The puzzle starts with disks sorted in ascending order of size from top to bottom (i.e., each disk sits on top of an even larger one). You have the following constraints:
>
> 1. Only one disk can be moved at a time.
> 2. A disk is slid off the top of one tower onto another tower.
> 3. A disk cannot be placed on top of a smaller disk.
>
> Write a recursive solution that prints every individual move in the format: `"Move disk X from Tower A to Tower C"`, and analyze the time complexity of your algorithm.

Implementation:

Listing 1: Tower of Hanoi (`problem1.py`)

```python
"""
CS361 - HW2
Problem 1 - Tower of Hanoi
Author: Shrey Poshiya
"""

def hanoi(n, source="A", auxiliary="B", target="C"):
    if n <= 0:
        return

    # move n-1 disk from source to auxiliary (target is now aux)
    hanoi(n - 1, source, target, auxiliary)
    # place the nth disk on the target rod
    print(f"move disk {n} from {source} to {target}")
    # move n-1 disk from auxiliary to target
    hanoi(n - 1, auxiliary, source, target)

def main():
    hanoi(3)

if __name__ == "__main__":
    main()
```

Sample output:

```
move disk 1 from A to C
move disk 2 from A to B
```

```
move disk 1 from C to B
move disk 3 from A to C
move disk 1 from B to A
move disk 2 from B to C
move disk 1 from A to C
```

Time Complexity Analysis: at every step of this recusion, we make two recurive calls, where each call is made to the $n-1$ disks aobve the nth disk. Therefore, when we get to the top disk, it's level in the recursive tree will have 2^n nodes. The next disk, will have 2^{n-1} nodes, and so on. Therefore, the total number of nodes (which is essentially the number of steps) in the recursive tree is: $2^n + 2^{n-1} + ... + 2^1 + 2^0$. Therefore,

$$O(2^n + 2^{n-1} + ... + 2^1 + 2^0) = O(2^n)$$

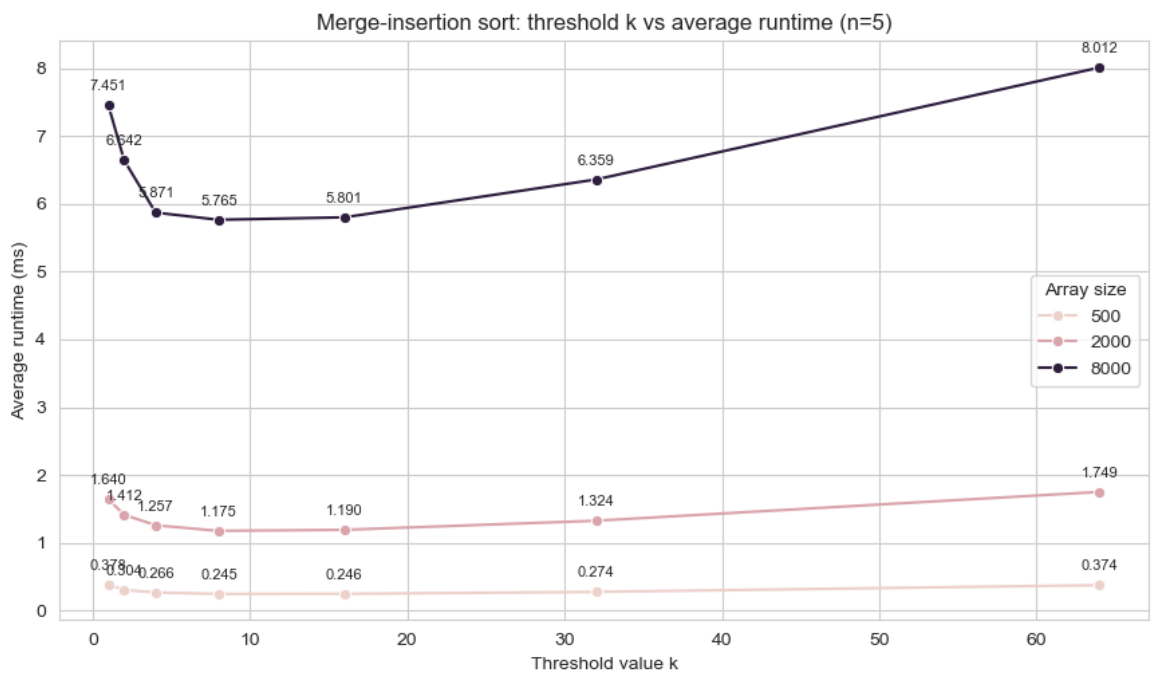
Problem 2

You've learned that merge sort has $O(n \log n)$ time complexity, but it has overhead from recursive calls. For small arrays, simpler algorithms like insertion sort can actually be faster.

Part A — Implementation:
Implement a hybrid sort that uses merge sort for large subarrays but switches to insertion sort when the subarray size falls below a threshold k.

Part B — Experimentation and Analysis:

1. Choose at least 3 different array sizes of your own between 100 and 10,000 (for example: 500, 2000, 8000). Pick sizes that run comfortably on your laptop.
2. For each array size, run your hybrid sort with k values of $1, 2, 4, 8, 16, 32, 64$.
3. For each combination of (n, k), repeat the sort 5 times on freshly generated random arrays and record the average runtime in milliseconds.
4. Plot your results with:
   - X-axis: threshold value k
   - Y-axis: average runtime in milliseconds
   - One curve per array size on the same plot
5. Identify the optimal value of k (lowest runtime) for each array size from your plot.
6. Explain why an optimal threshold exists, specifically, why both very small and very large values of k lead to worse performance.



1. The code to generate this plot is included in the Github link at the top of this page. Checkout **HW2/problem2.py** for the code.

2. The optimal value of k in each problem size is $k = 8$. In $N = 500$, the average runtime at this threshold is 0.245 ms. In $N = 2000$, the average runtime at this threshold is 1.175 ms. In $N = 8000$, the average runtime at this threshold is 5.765 ms.

3. In merge sort, as we get deeper into the tree, more recursive calls are required. For every level of the the merge sort tree, we need double the number of recursive calls as compared to the previous level. At low values of k, since the threshold is so low, we practically have to traverse the merge sort tree to almost the same number of tree levels as the un-modified merge sort tree. This means that we do not really save that many recursive calls. At high values of k, the inefficency of the insertion sort algorithm overpowers the overhead of the recursive calls from merge sort. In this case, we do not traverse many levels of the merge sort tree, and the inefficent merge sort is applied on relatviely large paritions of the array.

Problem 3

An **inversion** in an array is a pair of indices (i, j) where $i < j$ but $\texttt{arr}[i] > \texttt{arr}[j]$. This measures "how unsorted" an array is.

Task: Modify merge sort to count the total number of inversions in $O(n \log n)$ time.

Examples:

- $[1, 2, 3, 4]$ has 0 inversions (already sorted)
- $[4, 3, 2, 1]$ has 6 inversions (reverse sorted)
- $[2, 4, 1, 3]$ has 3 inversions: $(\texttt{arr}[0], \texttt{arr}[2]) = (2, 1)$, $(\texttt{arr}[1], \texttt{arr}[2]) = (4, 1)$, $(\texttt{arr}[1], \texttt{arr}[3]) = (4, 3)$, where each pair shows indices (i, j) with $i < j$ and $\texttt{arr}[i] >$

arr[j]

Questions:

1. What is the maximum number of inversions possible for an array of size n?
2. How does the inversion count relate to sorting algorithms like bubble sort?

Implementation:

Listing 2: Merge sort with inversion counting (`problem3.py`)

```python
"""
CS361 - HW2
Problem 3 - Counting Inversions
Author @Shrey Poshiya
"""

# merge sort implementation
def mergeSort(arr):
    """
    main merge sort function
    [arr] = list
    """
    if len(arr) <= 1:
        return (arr, 0)

    mid = len(arr) // 2

    lower_sorted, c1 = mergeSort(arr[:mid])
    higher_sorted, c2 = mergeSort(arr[mid:])
    merged, c3 = merge(lower_sorted, higher_sorted, 0)

    # overall the reccursive calls, continue adding up the inversions
    total_count = c1 + c2 + c3

    return merged, total_count


def merge(A,B,count):

    # if i were not using python list, C would be iitialized to be length of len(A) + len(B)
    C = []
    a = 0
    b = 0

    # iterate over the two lists and merge (this is done until one of the lists is completely merged)
    while (a < len(A)) and (b < len(B)):
        if A[a] < B[b]:
            C.append(A[a])
            a += 1
```

```python
        else:
            C.append(B[b])
            b += 1
            # iterate inversion count -> number of elements that B[b] skips over in A
            count += len(A) - a

    # if any elements missed from either, add them to the final list
    # first A then B to ensure order
    while (a < len(A)):
        C.append(A[a])
        a += 1

    while (b < len(B)):
        C.append(B[b])
        b += 1

    return C, count


def main():

    arr1 = [1,2,3,4]
    arr2 = [4,3,2,1]
    arr3 = [2,4,1,3]

    # sort each of the arrays

    sorted_arr1, inversion_1 = mergeSort(arr1)
    sorted_arr2, inversion_2 = mergeSort(arr2)
    sorted_arr3, inversion_3 = mergeSort(arr3)

    print(f"{arr1} -> after merge sort: {sorted_arr1} -> number of inversions = {inversion_1}")
    print(f"{arr2} -> after merge sort: {sorted_arr2} -> number of inversions = {inversion_2}")
    print(f"{arr3} -> after merge sort: {sorted_arr3} -> number of inversions = {inversion_3}")


if __name__ == "__main__":
    main()
```

Sample output:

```
[1, 2, 3, 4] -> after merge sort: [1, 2, 3, 4] -> number of inversions = 0
[4, 3, 2, 1] -> after merge sort: [1, 2, 3, 4] -> number of inversions = 6
[2, 4, 1, 3] -> after merge sort: [1, 2, 3, 4] -> number of inversions = 3
```

1. The maximum number of inversions possible for an array of size n will come when the list is completely reversed. In this case the number of inversions for each element (starting from the last) will be the number of elements before it: $(n-1)+(n-2)+...+1$. This can be

simplified to:

$$\sum_{i=0}^{n-1} i = \frac{n(n-1)}{2}$$

2. The inversion count is related to bubble sort (and other related sorting algorithms) because the number of inversions is equal to the number of swaps needed to sort the array in those algorithms.

Problem 4

Show how to implement a queue using two stacks.

We can have two stacks, call one the **auxiliary stack** and the other the **main stack**. When inserting an element into the queue, pop all elements from the main stack and push them into the auxiliary stack, one by one. Then push the incoming element into the main stack. Then pop all the elements from the auxiliary stack and push then back into the main stack. This method preserves the order of the elements in a queue formation in the main stack. When we want to de-queue an element, we can pop the top element of the main stack.

Problem 5

You are given the heads of two sorted linked lists `list1` and `list2`. Write a solution that merges the two lists into one sorted list. The list should be made by splicing together the nodes of the first two lists. Your function should return the head of the merged linked list.

Example:
Input: `list1 = [1,2,4]`, `list2 = [1,3,4]`
Output: `[1,1,2,3,4,4]`

Implementation:

Listing 3: Merge two sorted linked lists (`problem5.py`)

```python
"""
CS361 - HW2
Problem 5 - Merge Two Sorted Linked Lists
Author @Shrey Poshiya
"""

# helper classes for the linked list
class Node:
    def __init__(self, value):
        self.value = value
        self.next = None


class LinkedList:
    def __init__(self):
        self.head = None

    def build_list(self, arr):
        for value in arr:
```

```python
            self.insert(value)

    def insert(self, value):
        new_node = Node(value)
        if self.head is None:
            self.head = new_node
        else:
            current = self.head
            # travese the linked list
            while current.next is not None:
                current = current.next

            # add new node
            current.next = new_node

    def printList(self):
        current = self.head
        while current is not None:
            print(current.value, end=" -> ")
            current = current.next
        print("None")

def mergeLists(list1, list2):

    # initialize a new linked list:
    merged_list = LinkedList()

    # traverse the two lists
    current1 = list1.head
    current2 = list2.head

    # similar logic to merge sort: compare the values and
    # add smaller value to list
    # end traverasal when one of the lists is fully traversed
    while (current1 is not None) and (current2 is not None):
        if current1.value < current2.value:
            merged_list.insert(current1.value)
            print(f"inserting {current1.value} from list1")
            current1 = current1.next
        else:
            merged_list.insert(current2.value)
            print(f"inserting {current2.value} from list2")
            current2 = current2.next

    # if any elements missed, add them to final list
    while (current1 is not None):
        merged_list.insert(current1.value)
        current1 = current1.next

    while (current2 is not None):
        merged_list.insert(current2.value)
        current2 = current2.next

    return merged_list
```

```python

def main():

    list1 = [1,2,4]
    list2 = [1,3,4]

    list1_linked = LinkedList()
    list2_linked = LinkedList()

    list1_linked.build_list(list1)
    list2_linked.build_list(list2)

    merged_list = mergeLists(list1_linked, list2_linked)
    merged_list.printList()

if __name__ == "__main__":
    main()
```

Sample output:

```
inserting 1 from list2
inserting 1 from list1
inserting 2 from list1
inserting 3 from list2
inserting 4 from list2
1 -> 1 -> 2 -> 3 -> 4 -> 4 -> None
```

Problem 6

A **matrix diagonal** is a diagonal line of cells starting from some cell in either the topmost row or leftmost column and going in the bottom-right direction until reaching the matrix's end. Write a solution for a given $m \times n$ matrix of integers, sort each matrix diagonal in ascending order and return the resulting matrix.

Example:

$$\text{Input:} \quad \begin{pmatrix} 3 & 3 & 1 & 1 \\ 2 & 2 & 1 & 2 \\ 1 & 1 & 1 & 2 \end{pmatrix} \qquad \text{Output:} \quad \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & 2 & 2 & 2 \\ 1 & 2 & 3 & 3 \end{pmatrix}$$

Implementation:

Listing 4: Sort matrix diagonals (`problem6.py`)

```python
"""
CS361 - HW2
Problem 6 - Sort Matrix Diagonals
Author @Shrey Poshiya
"""

def sortMatrixDiagonals(matrix):

```

```python
    # only need to do sorting starting from the first elements from the first n-1 rows and m-1 columns

    numRows = len(matrix)
    numCols = len(matrix[0])

    # column logic:
    # iterate through the first n-1 columns
    for col in range(numCols - 1):

        # intiailzie iteration variables
        iterRow = 0
        iterCol = col
        diagonal = []
        # get the diagonal elements (iterates through the diagonal)
        while iterRow < numRows and iterCol < numCols:
            diagonal.append(matrix[iterRow][iterCol])
            iterRow += 1
            iterCol += 1

        # sort the diagonal list
        insertionSort(diagonal)

        # replace the diagonal elements in the matrix
        # reset iteration variables
        iterRow = 0
        iterCol = col
        for val in diagonal:
            matrix[iterRow][iterCol] = val
            iterRow += 1
            iterCol += 1

    # row logic:
    # iterate through the first m-1 rows
    # skip over row 0 since that was done in column log
    for row in range(1, numRows - 1):
        iterRow = row
        iterCol = 0
        diagonal = []
        # get the diagonal elements (iterates through the diagonal)
        while iterRow < numRows and iterCol < numCols:
            diagonal.append(matrix[iterRow][iterCol])
            iterRow += 1
            iterCol += 1

        # sort the diagonal list
        insertionSort(diagonal)

        # replace the diagonal elements in the matrix
        # reset iteration variables
        iterRow = row
        iterCol = 0
        for val in diagonal:
            matrix[iterRow][iterCol] = val
```

```python
            iterRow += 1
            iterCol += 1

# using insertion sort algo that I wrote from problem 2
def insertionSort(A):

    assert len(A) > 0, "error, ensure that provided list is NOT empty"

    for i in range(1, len(A)):
        # initialzie
        key = A[i]
        j = i - 1
        # shift up any elements that are greater than the key
        while j >= 0 and key <= A[j]:
            A[j + 1] = A[j]
            j  -= 1
        # move the key behind the shited up elements
        A[j+1] = key
    return A

def main():
    matrix = [[3,3,1,1],
              [2,2,1,2],
              [1,1,1,2]]

    print(f"Original matrix:")
    for row in matrix:
        print(row)

    sortMatrixDiagonals(matrix)

    print(f"Sorted matrix:")
    for row in matrix:
        print(row)

if __name__ == "__main__":
    main()
```

Sample output:

```
Original matrix:
[3, 3, 1, 1]
[2, 2, 1, 2]
[1, 1, 1, 2]
Sorted matrix:
[1, 1, 1, 1]
[1, 2, 2, 2]
[1, 2, 3, 3]
```

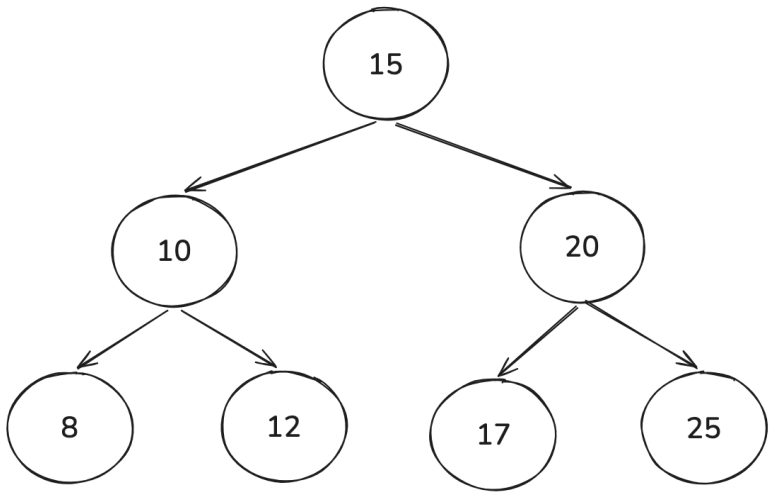
Problem 7

Consider the following sequence of numbers: $15, 10, 20, 8, 12, 17, 25$.

(a) Insert these numbers in the given order into a Binary Search Tree (BST). Draw the resulting tree.

(b) Identify whether the constructed BST is also a **Complete Binary Tree** and a **Full Binary Tree**. Justify your answer.

(c) What is the in-order traversal of the BST you constructed? What do you notice about the output?



1. The resulting tree is shown above.
2. A **Full Binary Tree** is a binary tree in which all nodes are either completely filled or completely empty. The above binary tree is a full binary tree because all nodes show this behavior.
3. A **Complete Binary Tree** is a binary tree in which all levels are completely filled except for the last level (left aligned). In the above tree, we see that all levels are completely filled; therefore, by default it is a complete binary tree.
4. The in-order traversal of the BST is: $8, 10, 12, 15, 17, 20, 25$.